VALIDACIÓN DE ESQUEMAS Y DOCUMENTOS

Presentación

En el desarrollo de aplicaciones modernas es común utilizar bases de datos NoSQL como MongoDB debido a su flexibilidad y escalabilidad. Sin embargo, esa flexibilidad no significa ausencia de reglas. Es indispensable establecer mecanismos de validación que aseguren la consistencia de la información almacenada. MongoDB permite definir reglas de validación mediante JSON Schema, permitiendo controlar tipos de datos, valores permitidos, campos obligatorios, tamaños mínimos y máximos, expresiones regulares, entre muchas otras restricciones.

¿Qué es una validación?

Es un conjunto de reglas que deben cumplir los datos antes de ser almacenados.

Su propósito es evitar:

- Información incompleta
- Datos con tipos incorrectos
- Valores fuera del rango permitido
- Errores de digitación

- Inconsistencia entre documentos

¿Qué es JSON Schema?

Es un estándar que permite describir la estructura que debe cumplir un documento JSON.

MongoDB implementa JSON Schema para validar documentos.

```JSON
{
   "nombre":"Carlos",
   "edad":25,
   "correo":"correo@gmail.com"
}
```

Ventajas de utilizar validaciones

- Mejor calidad de datos.
- Evita errores de programación.
- Mayor seguridad.
- Integridad de la información.
- Facilita mantenimiento del sistema.
- Reduce errores humanos.

Tipos de validaciones

Tipo de dato

```JSON
{
   "bsonType":"string"
}
```

Campos obligatorios

```JSON
"required":[
"nombre",
"correo"
]
```

Longitud Mínima

```JSON
{
   "minLength":5
}
```

Longitud Máxima

```JSON
{
   "maxLength":50
}
```

Valor Mínimo

```JSON
{
   "minimum":18
}
```

Valor Máximo

```JSON
{
   "maximum":100
}
```

Valores permitidos

```JSON
{
   "enum":[
      "Activo",
      "Inactivo"
   ]
}
```

Expresiones regulares

Validación de correo electrónico

```JSON
{
"pattern":"^.+@.+\\..+$"
}
```

Arreglos

```JSON
{
"bsonType":"array"
```

Objetos

```JSON
{
"bsonType":"object"
}
```

Sintaxis general

```javascript
db.createCollection("usuarios",{

validator:{

$jsonSchema:{

bsonType:"object",

required:[
"nombre",
"edad"
],

properties:{

nombre:{

bsonType:"string"

},

edad:{

bsonType:"int"

}

}

}

}

})
```

Ejemplo

Crear colección Estudiantes

```
db.createCollection("Estudiantes",{
validator:{
        $jsonSchema:{
                bsonType:"object",
                required:[ "nombre", "edad", "correo"],
                properties:{
                        nombre:{
                                bsonType:"string",
                                minLength:5
                        },
                        edad:{
                                bsonType:"int",
                                minimum:15,
                                maximum:60
                        },
                        correo:{
                                bsonType:"string",
                                pattern:"^.+@.+\\..+$"
                        }  //correo
                } //properties
        }  //$jsonchema
}  //validator
})  //createcollection
```

Modificar un esquema existente

```javascript
db.runCommand({
collMod:"estudiantes",
validator:{
$jsonSchema:{
...
}
}
})
```

Niveles de validación

Strict: Valida todos los documentos.

```javascript
validationLevel:"strict"
```

Moderate: Solo valida documentos nuevos.

```javascript
validationLevel:"moderate"
```

Buenas prácticas

- Definir siempre campos obligatorios.
- Utilizar tipos BSON correctos.
- Validar correos electrónicos.
- Limitar tamaños de cadenas.
- Utilizar valores mínimos y máximos.
- Evitar documentos excesivamente grandes.
- Documentar las reglas implementadas.
- Mantener consistencia entre colecciones.

Reglas de validación del esquema de MongoDB

Para conseguir que todos los documentos cumplan con los campos anteriores, utilizaremos un esquema específico de **MongoDB**. Se puede pensar que un esquema MongoDB no es más que un conjunto de reglas para las propiedades (Keys) y valores de los documentos. Dichas reglas funcionan en función de cada colección. Las reglas deben seguirse (=validated) durante la inserción o actualización de cada documento en la colección específica. Este conjunto de reglas debe definirse mediante un archivo **JSON** de acuerdo con los estándares **BSON**.

La validación de esquemas y documentos permite crear criterios que deben cumplir los documentos para poder ser introducidos o actualizados en una colección. Para llevarlo a cabo se utilizan validadores con los que describimos pautas que deben cumplir los documentos en lenguaje MQL como si se tratase del criterio de una búsqueda o actualización en MongoDB.
Para la validación del esquema se utiliza el operado**r $jsonChema,** como se muestra en el ejemplo:

Ejemplo de un objeto jsonSchema

```
{    $jsonSchema: {
                        required: ['customer'], // the customer field is required
                        properties: {
                                purchaseMethod: {
                                 enum: ['In Store','Online'],
                                description: "can only be either 'In Store' or 'Online'"
                                },
                                customer: {
                                        bsonType: "string",
                                        description: "must be a string"
                                },
                        }
                }
}
```

El operador **$jsonSchema** admite varias palabras clave para especificar reglas de validación. Por ejemplo:

- El arreglo **required** define los campos obligatorios en su documento.
- El objeto **properties** define reglas para campos específicos del documento.

Considere el siguiente ejemplo de validación:

```
{
  $jsonSchema: {
    bsonType: "object",
    required: [ "name", "year", "major", "gpa" ],
    properties: {
      name: {
        bsonType: "string",
        description: "must be a string"
      },
      year: {
        bsonType: "int",
        minimum: 2017,
        maximum: 3017,
        exclusiveMaximum: false,
```

```
            description: "must be an integer in [ 2017, 3017 ]"
          },
          major: {
            bsonType: "string",
            enum: [ "Math", "English", "Computer Science", "History", null ],
            description: "can only be one of the enum values"
          },
          gpa: {
            bsonType: [ "double" ],
            minimum: 0,
            description: "must be a double"
          }
        }
      }
}
```

De acuerdo al ejemplo anterior encontramos las siguientes reglas:

- Lista de **campos obligatorios**
- El **bsonType** para todos los campos
- Los valores mínimo y máximo para el año
- Los valores aceptables para la especialidad (**major**), utilizando **enum**.
- El valor mínimo para el campo de promedio general (**gpa**)

¿Cómo aceptar valores null a los atributos?

Ejemplo validación de un atributo llamado storeLocation

```
{
"$jsonSchema": {
        "properties": {
                "storeLocation": { "bsonType": [ "null", "string" ] }
        }
}
```

Validar campos de tipo array

Ejemplo validación de un atributo llamado location

```
"location": {
        "bsonType": "array",
         "description": "Location must be an array of strings",
         "minItems": 1,
         "uniqueItems": true,
         "items": {
              "bsonType": "string"
          }
  }
```

Validación usando operadores de consulta

También se puede especificar la validación mediante operadores de consulta, con la excepción de los siguientes operadores de consulta: $near, $nearSphere, $text y $where.

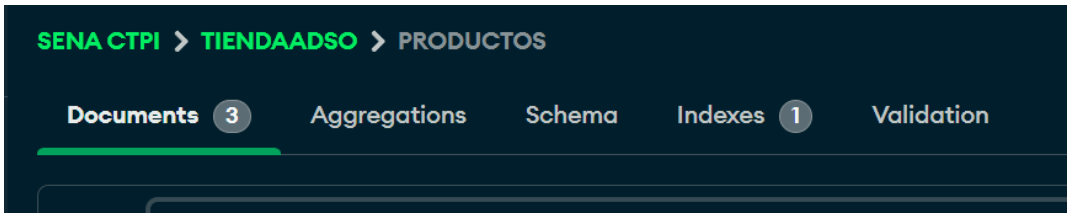
```
{
  $or: [
    { telefono: { $type: "string" } },
    { correo: { $regex: /@mongodb\.com$/ } },
    { estado: { $in: [ "desconocido", "Incompleto" ] } }
  ]
}
```

Para utilizar esta validación, debe cumplirse **una de las siguientes** condiciones:

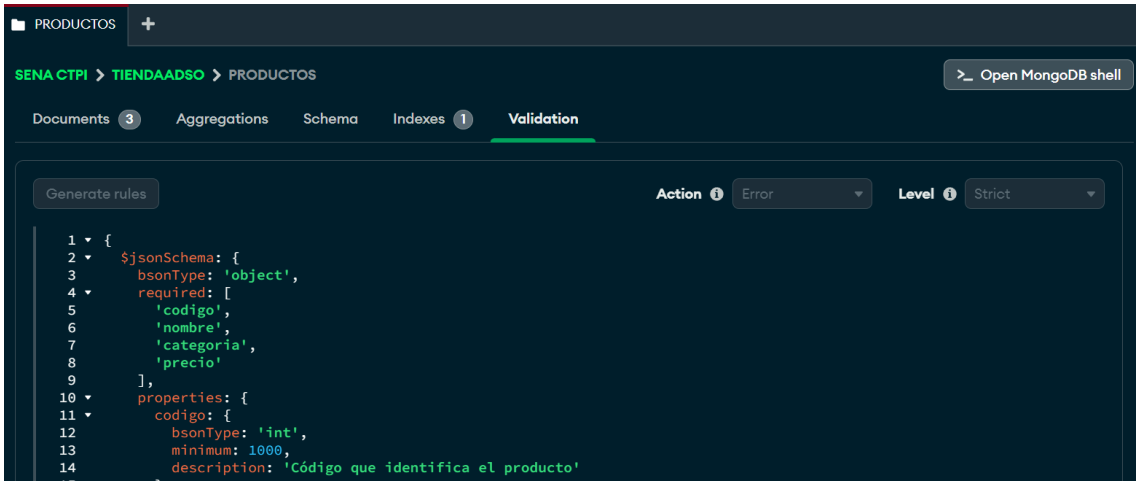
- El campo de teléfono debe ser una cadena de tipo BSON,
- El campo de correo electrónico debe coincidir con la expresión regular /@mongodb\.com$/, o
- El campo de estado debe ser desconocido o incompleto

Acciones y niveles de validación

En la interfaz gráfica de MongoDB Compass seleccione una colección y en la parte superior aparece unas opciones como se muestra a continuación



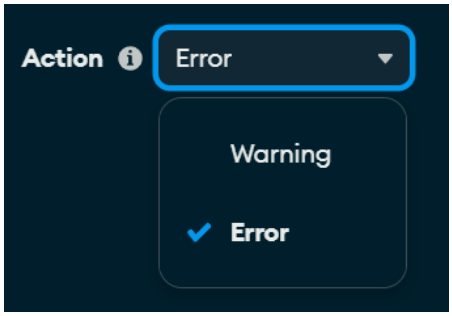
Aquí podemos dar clic en **Validation** donde aparece la siguiente interfaz:

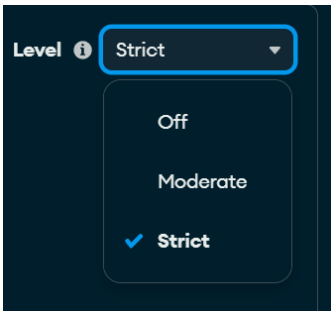


La **acción de validación** determina si se debe advertir pero aceptar documentos no válidos, o generar un error y rechazar documentos no válidos.

- **validateAction**: puede ser ‘error’ o ‘warning’

- El **nivel de validación** determina con qué rigurosidad MongoDB aplica las reglas de validación a los documentos existentes.
  - La validación estricta aplica sus reglas a todas las inserciones y actualizaciones de documentos.
  - La validación moderada solo aplica sus reglas a los documentos nuevos y a los documentos válidos existentes. Los documentos no válidos existentes no se ven afectados.
  - **validaLevel**: puede ser ‘moderate’ o ‘strict’





Para obtener más información sobre las acciones y los niveles de validación, consulte Especificar reglas de validación en el manual de MongoDB.

De acuerdo con lo anterior vamos a realizar el siguiente ejemplo desde la consola utilizando la herramienta **mongosh** y creando una base de datos en **MongoAtlas**. Para el ejercicio revisar la cadena de conexión a su base de datos en Atlas creada la sesión anterior y sino ingresar a Atlas y obtenerla.

Pasos:

- Abrir una terminal y ejecutar **mongosh cadenaconexionAtlas**
- Crear la base de datos llamada **NEGOCIOADSO_MIAPELLIDO**
- Crear la colección **PRODUCTOS_MINOMBRE** con los siguientes atributos y reglas.
    - **ATRIBUTOS**
        - **Codigo**: de tipo entero, valor mínimo 1000
        - **Nombre**: de tipo string
        - **Categoria**: de tipo string, con solo las siguientes posibilidades: Electrodomesticos, Ropa, Calzado.
        - **Precio**: de tipo double, valor mínimo 0.
- Todos los **atributos** son **obligatorios**
- A todos los **atributos** colocarles una **descripción**.

Solución:

1. Crear la base de datos

```
Atlas atlas-he7cfz-shard-0 [primary] test> use NEGOCIOADSO_CUELLAR
switched to db NEGOCIOADSO_CUELLAR
Atlas atlas-he7cfz-shard-0 [primary] NEGOCIOADSO_CUELLAR>
```

2. Crear el Json de la creación de la colección

```
db.createCollection("PRODUCTOS_CESAR",{
        validator: {
          $jsonSchema: {
            bsonType: "object",
            required: [ "codigo", "nombre", "categoria", "precio" ],
            properties: {
                        codigo: {
                          bsonType: "int",
                          minimum: 1000,
                          description: "Código que identifica el producto"
                        },
                        nombre: {
                          bsonType: "string",
                          description: "Nombre del producto ]"
                        },
                        categoria: {
                          bsonType: "string",
                          enum: [ "Electrodomestico", "Ropa", "Calzado" ],
                          description: "solo puede ser una de esas categorías"
                        },
                        precio: {
                          bsonType: "int",
                          minimum: 0,
                          description: "Precio del producto tipo entero"
                        }
            }
          }
        } ,
        validationLevel: "strict",
        validationAction: "error",

})
```

3. En la consola ejecutar la operación de crear la colección:

```
Atlas atlas-he7cfz-shard-0 [primary] NEGOCIOADSO_CUELLAR> db.createCollection("PRODUCTOS_CESAR",{
... validator: {
...    $jsonSchema: {
...       bsonType: "object",
...       required: [ "codigo", "nombre", "categoria", "precio" ],
...       properties: {
...          codigo: {
...             bsonType: "int",
...             minimum: 1000,
...             description: "Código que identifica el producto"
...          },
...          nombre: {
...             bsonType: "string",
...             description: "Nombre del producto ]"
...          },
...          categoria: {
...             bsonType: "string",
...             enum: [ "Electrodomestico", "Ropa", "Calzado" ],
...             description: "solo puede ser una de esas categorías"
...          },
...          precio: {
...             bsonType: "int",
...             minimum: 0,
...             description: "Precio del producto tipo entero"
...          }
...       }
...    }
... } ,
... validationLevel: "strict",
... validationAction: "error",
...
... })
{ ok: 1 }
Atlas atlas-he7cfz-shard-0 [primary] NEGOCIOADSO_CUELLAR>
```

4. Respuesta del servidor:



5. Realizar pruebas haciendo inserción de documentos

Insertar un documento vacío:

```
Atlas atlas-he7cfz-shard-0 [primary] NEGOCIOADSO_CUELLAR> db.PRODUCTOS_CESAR.insertOne({})
Uncaught:
MongoServerError: Document failed validation
Additional information: {
  failingDocumentId: ObjectId('6ade9448850b6203614d7944'),
  details: {
    operatorName: '$jsonSchema',
    schemaRulesNotSatisfied: [
      {
        operatorName: 'required',
        specifiedAs: { required: [ 'codigo', 'nombre', 'categoria', 'precio' ] },
        missingProperties: [ 'categoria', 'codigo', 'nombre', 'precio' ]
      }
    ]
  }
}
```

Genera error. Todos los atributos son requeridos.

Insertar otro documento con campo que no cumple la regla de validación.

{

'codigo': 500, 'nombre': 'Televisor',

'categoria': 'Electrodomestico', 'precio': 2500000

}

El producto anterior tiene código de 500 y debe ser como mínimo 1000.

```
Atlas atlas-he7cfz-shard-0 [primary] NEGOCIOADSO_CUELLAR> db.PRODUCTOS_CESAR.insertOne({ codigo: 500, nombre: 'Televis
or', categoria: 'Electrodomestico', precio: 2500000 })
Uncaught:
MongoServerError: Document failed validation
Additional information: {
  failingDocumentId: ObjectId('6ade95c6850b6203614d7947'),
  details: {
    operatorName: '$jsonSchema',
    schemaRulesNotSatisfied: [
      {
        operatorName: 'properties',
        propertiesNotSatisfied: [
          {
            propertyName: 'codigo',
            description: 'Código que identifica el producto',
            details: [
              {
                operatorName: 'minimum',
                specifiedAs: { minimum: 1000 },
                reason: 'comparison failed',
                consideredValue: 500
              }
            ]
          }
        ]
      }
    ]
  }
}
Atlas atlas-he7cfz-shard-0 [primary] NEGOCIOADSO_CUELLAR>
```

Ahora cambiemos el código por un valor valido

```
Atlas atlas-he7cfz-shard-0 [primary] NEGOCIOADSO_CUELLAR>
 db.PRODUCTOS_CESAR.insertOne({ codigo: 1500, nombre: 'Te
levisor', categoria: 'Electrodomestico', precio: 2500000
})
{
  acknowledged: true,
  insertedId: ObjectId('6ade94f5850b6203614d7946')
}
```

Ahora vamos a intentar insertar un producto con una categoría no posible así:

```
Atlas atlas-he7cfz-shard-0 [primary] NEGOCIOADSO_CUELLAR> db.PRODUCTOS_CESAR.insertOne({
... codigo:1501,
... nombre: 'Nevera',
... categoria:'Comida',
... precio: 3150000})
Uncaught:
MongoServerError: Document failed validation
Additional information: {
  failingDocumentId: ObjectId('6ade9345850b6203614d7942'),
  details: {
    operatorName: '$jsonSchema',
    schemaRulesNotSatisfied: [
      {
        operatorName: 'properties',
        propertiesNotSatisfied: [
          {
            propertyName: 'categoria',
            description: 'solo puede ser una de esas categorías',
            details: [
              {
                operatorName: 'enum',
                specifiedAs: { enum: [ 'Electrodomestico', 'Ropa', 'Calzado' ] },
                reason: 'value was not found in enum',
                consideredValue: 'Comida'
              }
            ]
          }
        ]
      }
    ]
  }
}
```

Cambiamos categoría a Electrodomestico y debe permitir hacer la inserción.

```
Atlas atlas-he7cfz-shard-0 [primary] NEGOCIOADSO_CUELLAR>
db.PRODUCTOS_CESAR.insertOne({ codigo: 1501, nombre: 'Neve
ra', categoria: 'Electrodomestico', precio: 3150000 })
{
  acknowledged: true,
  insertedId: ObjectId('6ade96aa850b6203614d7948')
}
```

Consulta de los productos registrados

```
Atlas atlas-he7cfz-shard-0 [primary] NEGOCIOADSO_CUELLAR>
db.PRODUCTOS_CESAR.find()
[
  {
    _id: ObjectId('6ade94f5850b6203614d7946'),
    codigo: 1500,
    nombre: 'Televisor',
    categoria: 'Electrodomestico',
    precio: 2500000
  },
  {
    _id: ObjectId('6ade96aa850b6203614d7948'),
    codigo: 1501,
    nombre: 'Nevera',
    categoria: 'Electrodomestico',
    precio: 3150000
  }
```

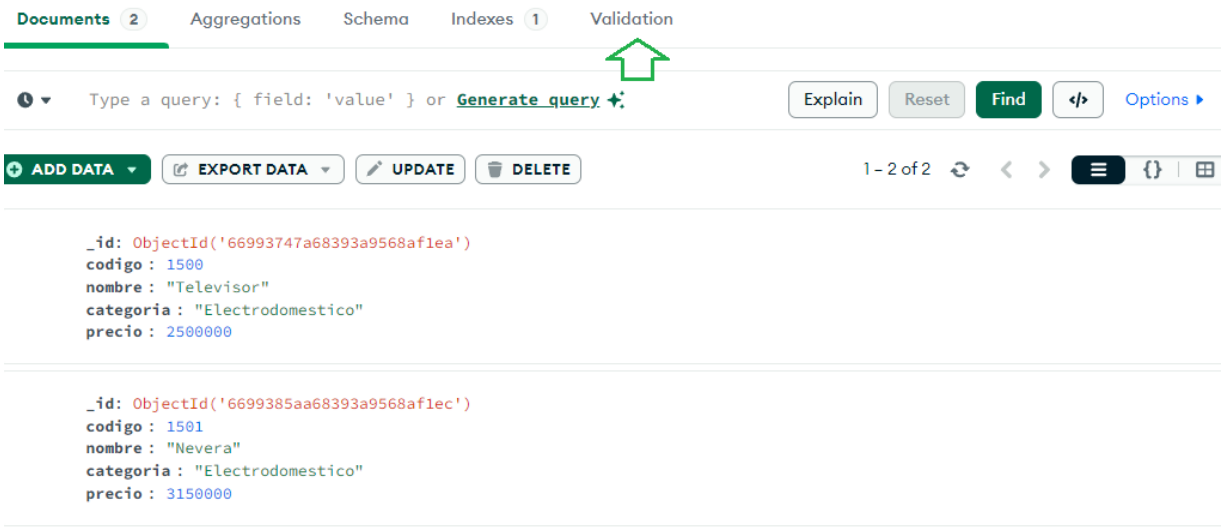
Desde la consola podemos revisar información de las reglas de validación de la colección **PRODUCTOS_CESAR**

```
Atlas atlas-he7cfz-shard-0 [primary] NEGOCIOADSO_CUELLAR> db.getCollectionInfos()
[
  {
    name: 'PRODUCTOS_CESAR',
    type: 'collection',
    options: {
      validator: {
        '$jsonSchema': {
          bsonType: 'object',
          required: [ 'codigo', 'nombre', 'categoria', 'precio' ],
          properties: {
            codigo: {
              bsonType: 'int',
              minimum: 1000,
              description: 'Código que identifica el producto'
            },
            nombre: {
              bsonType: 'string',
              description: 'Nombre del producto ]'
            },
            categoria: {
              bsonType: 'string',
              enum: [ 'Electrodomestico', 'Ropa', 'Calzado' ],
              description: 'solo puede ser una de esas categorías'
            },
            precio: {
              bsonType: 'int',
              minimum: 0,
              description: 'Precio del producto tipo entero'
            }
          }
        }
      },
      validationLevel: 'strict',
      validationAction: 'error'
    },
    info: {
      readOnly: false,
      uuid: UUID('ec071287-9331-4662-841a-82020dd4939e')
    },
    idIndex: { v: 2, key: { _id: 1 }, name: '_id_' }
```

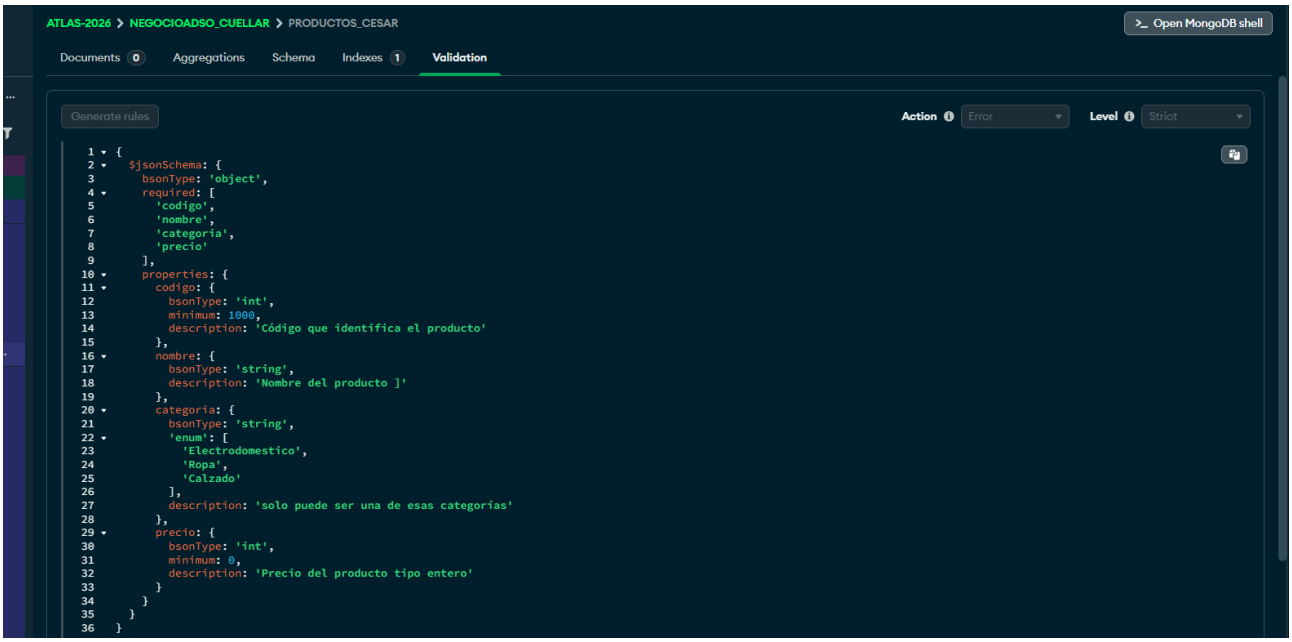
```
Atlas atlas-he7cfz-shard-0 [primary] NEGOCIOADSO_CUELLAR> db.getCollectionInfos({name:'PRODUCTOS_CESAR'})[0]
{
  name: 'PRODUCTOS_CESAR',
  type: 'collection',
  options: {
    validator: {
      '$jsonSchema': {
        bsonType: 'object',
        required: [ 'codigo', 'nombre', 'categoria', 'precio' ],
        properties: {
          codigo: {
            bsonType: 'int',
            minimum: 1000,
            description: 'Código que identifica el producto'
          },
          nombre: { bsonType: 'string', description: 'Nombre del producto ]' },
          categoria: {
            bsonType: 'string',
            enum: [ 'Electrodomestico', 'Ropa', 'Calzado' ],
            description: 'solo puede ser una de esas categorías'
          },
          precio: {
            bsonType: 'int',
            minimum: 0,
            description: 'Precio del producto tipo entero'
          }
        }
      }
    },
    validationLevel: 'strict',
    validationAction: 'error'
  },
  info: {
    readOnly: false,
    uuid: UUID('ec071287-9331-4662-841a-82020dd4939e')
  },
  idIndex: { v: 2, key: { _id: 1 }, name: '_id_' }
```

También se puede consultar las reglas de validación desde la interfaz gráfica.

Debe seleccionar la colección y en la parte principal donde se muetran los datos dar clic en la opción validation como lo muestra la siguiente imagen:



Después aparece la información de las reglas de validación de la colección así:



Desde la interfaz anterior se pueden hacer cambios a las reglas de validación y la interfaz cuando usted realiza un cambio se habilita un botón para actualizar.

Referencias

- **Reglas de validación del esquema de MongoDB**: https://blog.itechnova.com/reglas-de-validacion-del-esquema-de-mongodb/
- **Validación Nativa en MongoDB**: https://www.paradigmadigital.com/dev/validacion-nativa-documentos-mongodb-3-2/
- **Validación de esquemas:** https://www.youtube.com/watch?v=t-Z3xIcrMqg
- https://docs.mongodb.com/compass/current/validation/
- https://docs.mongodb.com/manual/core/schema-validation/#schema-validation
- https://docs.mongodb.com/manual/core/schema-validation/#specify-validation-rules
- https://docs.mongodb.com/manual/reference/command/collMod/#add-document-validation-to-an-existing-collection
- https://docs.mongodb.com/manual/reference/bson-types/
- https://www.mongodb.com/docs/manual/reference/operator/query/jsonSchema/

CONTROL DEL DOCUMENTO

	Nombre	Cargo	Dependencia	Fecha
Autor (es)	César Marino Cuéllar Chacón	Instructor	CTPI-CAUCA	27-10-2026